Project: Auto-scaling Data Access and Business Logic

Due: 04/25/2026 12:59 AM
Extended Deadline: 04/29/2026 10:53 AM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Single Instance - MySQL
- Task Overview
- Task

Separate Reader + Writer for MySQL
- Task Overview
- Task

Horizontal Autoscaling
- Task Overview
- Task

Wrap Up
- Conclusion

Separate Reader + Writer for MySQL (Task)

Scaling Up Our System

Intro

With your login service now backed by a persistent MySQL database, the next bottleneck becomes clear — a single database pod means every read and write query hits the same instance. Under load, this becomes a chokepoint that no amount of application-level scaling can fix. In this task, you will scale your database layer by replacing the single MySQL pod with a replicated StatefulSet: one writer and two read replicas. You will then configure Django to route read and write queries to the appropriate database, and measure the performance difference using Locust. This is how production systems handle database scale — not by making one database faster, but by distributing the load across many.

Setup Locust

To measure the impact of our database scaling, we need a way to simulate real traffic against our system. **Locust** is a load testing tool that lets us do exactly that — it will send a stream of requests to our Django app so we can compare how many requests per second our system can handle before and after adding read replicas. Without this, we'd have no way to quantify whether our scaling changes actually made a difference.

Let's quickly get a rough idea of how well our system load-tests using **Locust**. Create a folder named **t2-locust** the repo's root directory using:

```
poetry new t2-locust
cd t2-locust
poetry add locust
```

Inside the `t2-locust` directory, create a folder named `t2_locust`. Then inside that folder, create a file named `locustfile.py`:

```
t2-locust/
└── t2_locust/
    └── locustfile.py
```

```
from locust import HttpUser, task
class QuickstartUser(HttpUser):
    @task
    def hello_world(self):
```

```
        data_dict = {
          "username":"admin",
          "password":"asdasd"
        }
        self.client.post("/api/token/", data_dict)
```

You can then run a loadtest with something like:

```
poetry run locust -f t2_locust/locustfile.py --headless -u
```

You will likely see something like the following:

```
Type     Name
--------|-----------------------------------------------------
POST     /api/token/
--------|-----------------------------------------------------
         Aggregated
```

While this **service** really does not need to be heavily optimized for this application, as the LLM inference is the bottle-neck, we can see that we achieve about 2 requests per second at best, with an average response latency of about 5 seconds. This does make some sense, as our system does need to do some cryptographic operations, but currently we are fairly bottlenecked by our existing system, as we only **have a single database pod** to work with.
Before we go and optimize, however, let's quickly confirm that this is the issue we need to solve. We can do that by **increasing** the number of replicas to 3 in our **Django deployment**.
After re-running our pod, we will see that we get the following:

```
Type     Name
--------|-----------------------------------------------------
POST     /api/token/
--------|-----------------------------------------------------
         Aggregated
```

This is already **significantly better**. We have a system where our average latency is about **2** seconds now, and we can scale up to 5x the number of requests. However, this is still **quite** slow. After all, we're literally only getting a single response from our database - we would expect a faster scaling.
For that reason, we will need to **replicate** our **MySQL** pod. **Replication** is a best-practice where we **literally copy** the contents of our database to multiple systems. In our case, we will use **leader-follower replication**, where we only have a single **pod** writing to the database and all other pods can read from it. This generally **simplifies** the workload, and makes it far easier for our system to scale up.
Remove the task 1 MySQL resources from Git (HelmRelease and related manifests under `k8s/`, `helm-releases/`, etc.) so **Flux** can prune the old single-pod MySQL stack before you add the replicated setup.

Subtask 1: Setting up Read-Write Replication

Now, we could make you write most of this code by hand. However, in this case, we'll walk you through the configuration side of this, as this course is

less about "learning how to manage the specifics of a MySQL database" and more "what do you need to think about when you do this". We're really just following [this wonderful resource](#) provided by the Kubernetes documentation, which does a reasonable job of discussing replication.

First, add the following **ConfigMap** under `k8s/mysql/` (for example `k8s/mysql/mysql-config.yaml`), include it in your `kustomization.yaml` if you use one, **commit and push**, and let **Flux** reconcile it:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mysql-config
data:
  master.cnf: |
    [mysqld]
    log-bin=mysql-bin
    binlog-format=ROW
    server-id=1
  slave.cnf: |
    [mysqld]
    server-id=2
  init-master.sh: |
    #!/bin/bash
    set -ex
    echo "Creating replication user..."
    mysql -h 127.0.0.1 -u root -p${MYSQL_ROOT_PASSWORD} <<I
    CREATE USER IF NOT EXISTS 'replication'@'%' IDENTIFIED
    GRANT REPLICATION SLAVE ON *.* TO 'replication'@'%';
    FLUSH PRIVILEGES;
    EOF
  init-slave.sh: |
    #!/bin/bash
    set -ex
    echo "Waiting for master to be ready..."
    until mysql -h 127.0.0.1 -h mysql-0.mysql -u root -p${M
      echo "Master is not ready yet..."
      sleep 3
    done
    echo "Getting master position..."
    MASTER_STATUS=$(mysql -h mysql-0.mysql -u root -p${MYS(
    MASTER_LOG_FILE=$(echo $MASTER_STATUS | cut -f 1 -d '
    MASTER_LOG_POS=$(echo $MASTER_STATUS | cut -f 2 -d ' ';
    echo "Stopping replica IO thread if running..."
    mysql -h 127.0.0.1 -u root -p${MYSQL_ROOT_PASSWORD} -e
    echo "Setting up slave with master log file: $MASTER_LO
    mysql -h 127.0.0.1 -u root -p${MYSQL_ROOT_PASSWORD} <<I
    CHANGE MASTER TO
    MASTER_HOST='mysql-0.mysql',
    MASTER_USER='replication',
    MASTER_PASSWORD='repl_password',
    MASTER_LOG_FILE='$MASTER_LOG_FILE',
    MASTER_LOG_POS=$MASTER_LOG_POS;
    START SLAVE;
```

```
    EOF
    mysql -h 127.0.0.1 -u root -p${MYSQL_ROOT_PASSWORD} <<E
    SET GLOBAL super_read_only = 1
    EOF
    echo "Slave setup complete!"
```

All this does is configure the database replicas to **only be read-focused** whereas the **primary leader** can be write-focused. This does speed up performance, as the database can now make use of the fact that a couple of the replicas are read-only as opposed to write-and-read focused, and can have significant performance increases.

Besides having separate configuration, we will also **need to separate read and write queries** from the service-side.
Under **k8s/mysql/**, create a `mysql-services.yaml` file and define the following services:

```
# Define this service first, which will ensure writes in ou
apiVersion: v1
kind: Service
metadata:
  name: mysql
  labels:
    app: mysql
spec:
  ports:
  - port: 3306
    name: mysql
  clusterIP: None
  selector:
    app: mysql
---

#For all other pods, we will instead use the following, wh
apiVersion: v1
kind: Service
metadata:
  name: mysql-read
  labels:
    app: mysql
    app.kubernetes.io/name: mysql
    readonly: "true"
spec:
  ports:
  - name: mysql
    port: 3306
  selector:
    app: mysql
    replica: "true"
```

As we have a system **where we need to separate the leader pod** from the followers, and one where **each pod must be with the same volume upon restart** to ensure performance, we need to do something slightly differently than what we have done before - instead of defining a **Deployment** resource, we will instead **define a StatefulSet**. A **StatefulSet** is a particular workload,

like a **ReplicaSet** or **Deployment**. Unlike **Deployments**, however, **StatefulSet** pods **have some state** associated with them. This allows **Kubernetes** to separate each system as expected, and make sure that the pods are behaving as you would expect.

Next, add the following **mysql-statefulset** manifest under `k8s/mysql` (for example `k8s/mysql/mysql-statefulset.yaml`), wire it into your Kustomize bundle, **commit and push**, and let **Flux** reconcile it:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  selector:
    matchLabels:
      app: mysql
  serviceName: mysql
  replicas: 3
  template:
    metadata:
      labels:
        app: mysql
    spec:
      initContainers:
      - name: init-mysql
        image: mysql:8.0
        command:
        - bash
        - "-c"
        - |
          set -ex
          # Generate server-id based on ordinal index
          [[ $HOSTNAME =~ -([0-9]+)$ ]] || exit 1
          ordinal=${BASH_REMATCH[1]}
          # Only the first pod (ordinal 0) is master
          if [[ $ordinal -eq 0 ]]; then
            cp /mnt/config-map/master.cnf /etc/mysql/conf.d
          else
            # Update server-id for slaves
            cp /mnt/config-map/slave.cnf /etc/mysql/conf.d/
            sed -i "s/server-id=2/server-id=$((ordinal + 1)
          fi
        volumeMounts:
        - name: conf
          mountPath: /etc/mysql/conf.d
        - name: config-map
          mountPath: /mnt/config-map
      containers:
      - name: mysql
        image: mysql:8.0
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: "rootpassword"
        ports:
```

```
          - name: mysql
            containerPort: 3306
          volumeMounts:
          - name: data
            mountPath: /var/lib/mysql
          - name: conf
            mountPath: /etc/mysql/conf.d
          - name: config-map
            mountPath: /mnt/config-map
          resources:
            requests:
              cpu: 500m
              memory: 1Gi
          livenessProbe:
            exec:
              command: ["mysqladmin", "ping", "-u", "root", '
            initialDelaySeconds: 30
            periodSeconds: 10
            timeoutSeconds: 5
          readinessProbe:
            exec:
              command: ["mysql", "-u", "root", "-prootpasswo
            initialDelaySeconds: 5
            periodSeconds: 2
            timeoutSeconds: 1
        - name: replication-init
          image: mysql:8.0
          command:
          - bash
          - "-c"
          - |
            set -ex
            # Wait for MySQL to be ready
            until mysqladmin ping -h 127.0.0.1 -u root -p${M
              echo "Waiting for MySQL to be ready..."
              sleep 2
            done
            # Determine if master or slave based on hostname
            [[ $HOSTNAME =~ -([0-9]+)$ ]] || exit 1
            ordinal=${BASH_REMATCH[1]}
            if [[ $ordinal -eq 0 ]]; then
              echo "Initializing master..."
              # Execute the script content instead of trying
              bash -c "$(cat /mnt/config-map/init-master.sh)'
            else
              echo "Initializing slave..."
              # Execute the script content instead of trying
              bash -c "$(cat /mnt/config-map/init-slave.sh)"
            fi
            # Keep container running
            tail -f /dev/null
          env:
          - name: MYSQL_ROOT_PASSWORD
            value: "rootpassword"
          volumeMounts:
```

```
          - name: config-map
            mountPath: /mnt/config-map
        volumes:
        - name: conf
          emptyDir: {}
        - name: config-map
          configMap:
            name: mysql-config
    volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 10Gi
```

> **Information**
>
> Note that we're switching from **password** to **rootpassword**

While it looks **intimidating**, in practice it's really not doing too much more than what we have already talked about:

1. We have an **initContainers** container that just **copies over** the correct **conf.d** file for each pod, using the **HOSTNAME** to get the pod's index.
2. Our volume mounts are fairly similar - all we are doing is just ensuring that the **config-map** is copied over.
3. We use a `replication-init` sidecar container that waits for the MySQL container to become ready, then executes either the `init-master` or `init-slave` script depending on the role.
4. We create some special configurations to **allow our system to update the replicas dynamically**. We are relying on the default replication mechanism provided by MySQL.
   The commands given are scary, but the overall process is quite similar to what you have done already. All that we get with a **StatefulSet** is the added flexibility that sequentially starting pods provides, and the extra persistence necessary to make sure that we do have everything that we expect.

Finall, create a kustomization.yaml that'll install the resources in the following order:

```
- mysql-config.yaml
- mysql-services.yaml
- mysql-statefulset.yaml
```

The Default Namespace

Remember to add the default namespace to all resource definitions above.

Interim Validation

At the end, your directory structure for the k8s dir should look like this:

```
├── istio
│   ├── gateway.yaml
```

```
│   ├── kustomization.yaml
│   ├── namespace.yaml
│   ├── security.yaml
│   └── virtual-service.yaml
├── llm
│   ├── deployment.yaml
│   ├── kustomization.yaml
│   ├── model-pvc.yaml
│   └── service.yaml
├── login
│   ├── deployment.yaml
│   ├── kustomization.yaml
│   └── service.yaml
└── mysql
    ├── kustomization.yaml
    ├── mysql-config.yaml
    ├── mysql-services.yaml
    └── mysql-statefulset.yaml

5 directories, 16 files
```

Point FluxCD to the new `mysql` folder so it can reconcile the resources within it. You do this by creating a `mysql.yaml` file in the **my-cluster/kustomize-sources/** directory.

```
my-cluster/kustomize-sources/
├── cdpipeline.yaml
├── cipipeline.yaml
├── crds.yaml
├── helm-releases.yaml
├── helm-sources.yaml
├── istio.yaml
├── llm.yaml
├── login.yaml
└── mysql.yaml
```

Remove the Old MySQL Installation

Since we're switching to StatefulSet, we no longer need the mysql from task1. Go to the helm release's **kustomization.yaml** and comment out the `mysql.yaml`.

Delete the Old PVC

```
kubectl delete pvc data-mysql-0 -n default
```

Commit your Changes to Git

Once you're done with the changes, commit, push and reconcile your changes

If you check your pods:

```
kubectl get pods
```

You'll see that the **StatefulSet** pods are nicely named:

```
NAME                                READY   STATUS    RESTA
llmapp-deployment-7865d87cb-cx7rn   2/2     Running   0
login-7c7cd7885c-q77sw              2/2     Running   0
mysql-0                             3/3     Running   0
mysql-1                             3/3     Running   0
mysql-2                             3/3     Running   0
```

Check out the documentation to read more about StatefulSets - for this task you just need to get familiar enough with the YAML to make the changes necessary to get our application working.

Interim Validation

To check your progress, make sure that you have a database that looks like the original database setup, using:

```
kubectl run mysql-client --image=mysql:8.0 -it --rm --resta
```

You should get:

```
+------------------------+
| Database               |
+------------------------+
| information_schema     |
| mysql                  |
| performance_schema     |
| sys                    |
+------------------------+
```

Subtask 2: Configuring a Replicated Database

Now that our replicated StatefulSet is running, we need to:

1. Initialize the `login` database using a Kubernetes **Job**.
2. Label the replica pods so the `mysql-read` service can route read traffic to them.

Step 1: Create the Initialization Job

A **Kubernetes Job** is a one-off workload that runs to completion and then stops. It is commonly used for tasks like database initialization. Unlike a Deployment, it does not keep running after the task is done — Kubernetes will automatically retry it if it fails, up to the `backoffLimit`.

Create a Job manifest at `k8s/mysql/mysql-migration.yaml`:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: mysql-migration
  namespace: default
spec:
  template:
    spec:
      containers:
      - name: mysql-db-creator
        image: mysql:8.0
```

```
          command:
            - /bin/sh
            - -c
            - mysql -h mysql-0.mysql -uroot -prootpassword -e
        restartPolicy: Never
    backoffLimit: 4
```

Add it to your `kustomization.yaml`, then commit and push:

Flux will automatically reconcile and run the Job.

Step 2: Label the Replica Pods

The `mysql-read` service uses a `replica=true` selector to route read traffic only to the slave pods. Apply the label to `mysql-1` and `mysql-2`:

```
kubectl label pods mysql-1 replica=true
kubectl label pods mysql-2 replica=true
```

Interim Validation

Check that the `login` database was created on the master:

```
kubectl run mysql-client --image=mysql:8.0 -it --rm --resta
  mysql -h mysql-0.mysql -uroot -prootpassword -e 'SHOW DA
```

Then check that it replicated to the read replicas:

```
kubectl run mysql-client --image=mysql:8.0 -it --rm --resta
  mysql -h mysql-read -uroot -prootpassword -e 'SHOW DATABA
```

Both should return:

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| login              |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
```

Subtask3: Updating Django

Now we move onto the second part of this - updating Django. Django, by default, supports using separate databases - we just need to configure it so that it understands which databases are better for reading and which are better for writing.

In order to do this, we **will** need to pay extra attention to our settings.py file to configure our system accordingly.

First, we will need to modify **the settings.py** file for our system, to specify both our read and write replicas as **separate databases**.

Here, the **write replica** has a host of **mysql-0.mysql**, and **should be used as the default database**.

Secondly, we will need to develop a simple **database router** to ensure that

read queries are processed with the **read replicas** only.
We provide a sample **router class** for you - your only task is to **ensure that you connect it properly**:

```
class SimpleRouter:
    """
    A router to control all database operations on models :
    """
    def db_for_read(self, model, **hints):
        """
        Attempts to send read-only queries.
        """
        return SOMETHING.
```

For a rough tutorial, check out this post.
If you re-run the load-test now, you will see that the number of operations we can read has increased drastically, as expected.
For reference - we are able to get about 7 queries or so, which is dramatically more than just a simple replication.
Congrats! You have scaled up your system using replicas of an existing database.

Less-Scaffolded Subtask

This subtask is intentionally less guided. You will need to make changes across multiple files in your Django app — we won't tell you which ones. We expect you to read the instructions carefully, understand what needs to be configured, and come up with your own strategy to get it working.

Submission

Submit using:

```
./submitter -t task2
```

Reflection Questions

In the **answers/unit6answers.txt** file, respond to the following questions:

1. We hardcoded sensitive values like passwords directly in our manifests for the sake of simplicity. This is **never acceptable in production**. How would you store and inject these values securely in a real Kubernetes environment?
2. We scaled MySQL by adding read replicas, but scaling databases comes with its own set of challenges. What difficulties do you think arise when scaling MySQL in this way? Are there other database offerings or architectures that would handle scaling more gracefully?

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform